

RUANA — Manual Maestro

Red Unida de Apoyo entre Negocios Aliados

Campo	Valor
Documento	Manual Maestro (única fuente de verdad del proyecto)
Copia idéntica	README_RUANA_COMPLETO.md
Fecha de reorganización	2026-07-28
Versión documentada	0.9 (pre-MVP avanzado)
Fuente técnica	Código en RUANA/ (prioridad sobre cualquier .md previo)

> Principio de autoridad: si hay diferencia entre este manual y un documento en docs/archive/, prevalece este README. Si hay diferencia entre este README y el código, prevalece el código y el README debe actualizarse.

Índice navegable

1. Introducción
2. Modelo de negocio
3. Arquitectura
4. Estructura del repositorio
5. Reglas del negocio
6. Flujos completos
7. Roles
8. Estados
9. Base de datos
10. API
11. Seguridad
12. Variables de entorno
13. Panel de administración
14. Frontend
15. Despliegue y operaciones
16. Cómo ejecutar
17. Roadmap
18. Historial (changelog)
19. Índice de documentación secundaria
20. Notas de unificación documental

Documentación secundaria y archivo histórico: docs/README.md.

1. Introducción

1.1 Qué es RUANA

RUANA (Red Unida de Apoyo entre Negocios Aliados) es un sistema de control, coordinación y reputación profesional para redes locales de profesionales y pequeños negocios. Opera con reglas claras, consecuencias reales (score, competencia, purga) y trazabilidad completa en base de datos.

Lo que RUANA no es

- No es una red social.
- No es un marketplace abierto de volumen.
- No es un CRM tradicional.
- No prioriza crecimiento rápido a costa de calidad.

Lo que RUANA sí es

- Un sistema operativo de confianza para comunidades profesionales territoriales (por código postal).
- Memoria y consecuencias: el comportamiento se refleja en score, estado, competencia y, en última instancia,

expulsión.

- Presión positiva: el buen comportamiento se recompensa; el malo degrada el score.
- Orden profesional en un entorno donde las recomendaciones informales no tienen control de calidad.

1.2 Problema que resuelve

Hoy las recomendaciones entre profesionales no tienen consecuencias, no hay memoria operativa de errores y la confianza es frágil. RUANA introduce:

1. Memoria — historial persistente de aliados, contactos, chat, pagos y score.
2. Reglas operativas — score 0-100, estados derivados, contactos, Apoyo RUANA, competencia.
3. Presión positiva — recompensas y penalizaciones automáticas aplicadas por el backend.
4. Gobernanza territorial — grupos por código postal, plazas por oficio, suplentes, fusión/disolución.

1.3 Misión

Crear y sostener redes locales de profesionales donde la confianza sea medible, auditable y con consecuencias, de modo que recomendar y contratar entre aliados sea seguro para quien recomienda y justo para quien ejecuta.

1.4 Visión

Que cada zona (código postal) tenga grupos RUANA estables, con oficios no duplicados, reputación transparente y un flujo de encargos con chat, cierre económico y Apoyo a la red — sin depender de plataformas de volumen ni de reputación opaca.

1.5 Filosofía

Principio rector (no negociable)

- > «El panel no piensa. El score define el estado. El panel solo refleja estado.»
- El frontend presenta datos y recoge acciones.
- El backend (web/app.py + core/db_manager.py) valida, orquesta y aplica reglas.
- Las etiquetas DESTACADO / ESTABLE / EN RIESGO / COMPETENCIA se derivan del score; no se editan a mano en el panel.

Otros principios

1. Un aliado = una identidad: código único; el historial va ligado al código. Perder el código = perder acceso (no «crear otra cuenta» para resetear reputación).
2. Un código = una historia: no hay anonimato operativo.
3. Una base de datos = una verdad: Postgres/Supabase en producción; SQLite como fallback local/tests. Firebase Hosting no almacena el dominio de negocio.
4. Backend-first: el frontend no decide score, competencia ni cierre económico.
5. Compatibilidad legacy controlada: se aceptan códigos alfanuméricos históricos (A0001, ALFA01, ...); los nuevos aliados reciben código numérico de 5 dígitos.

2. Modelo de negocio

2.1 Cómo genera ingresos

El ingreso operativo documentado en código es el Apoyo RUANA:

- Al cerrar un encargo (trabajo_cerrado) con importe final, el sistema calcula:
$$\text{apoyo_ruana} = \text{importe_final} \times (\text{apoyo_pct} / 100)$$
- El porcentaje vigente se lee de RUANA/config/ruana_reglas_v1.json → clave apoyo_pct.
- Valor actual en configuración: 12.0 (12 %).
- El profesional debe abonar ese Apoyo (Bizum / IBAN / QR Revolut configurados en las mismas reglas). Sube comprobante → admin revisa → marca pagado o rechazado.
- La columna histórica ingresos_ruana.apoyo_ruana_2pct conserva el nombre legacy; el valor almacenado es el Apoyo calculado con apoyo_pct (no necesariamente 2 %).

> Unificación: documentación antigua citaba 15 %, 5 % o «2 %». El código y ruana_reglas_v1.json mandan: 12 % por defecto, editable por admin (POST /api/admin/cambiar-reglas / métodos de pago).

2.2 Reglas generales del producto

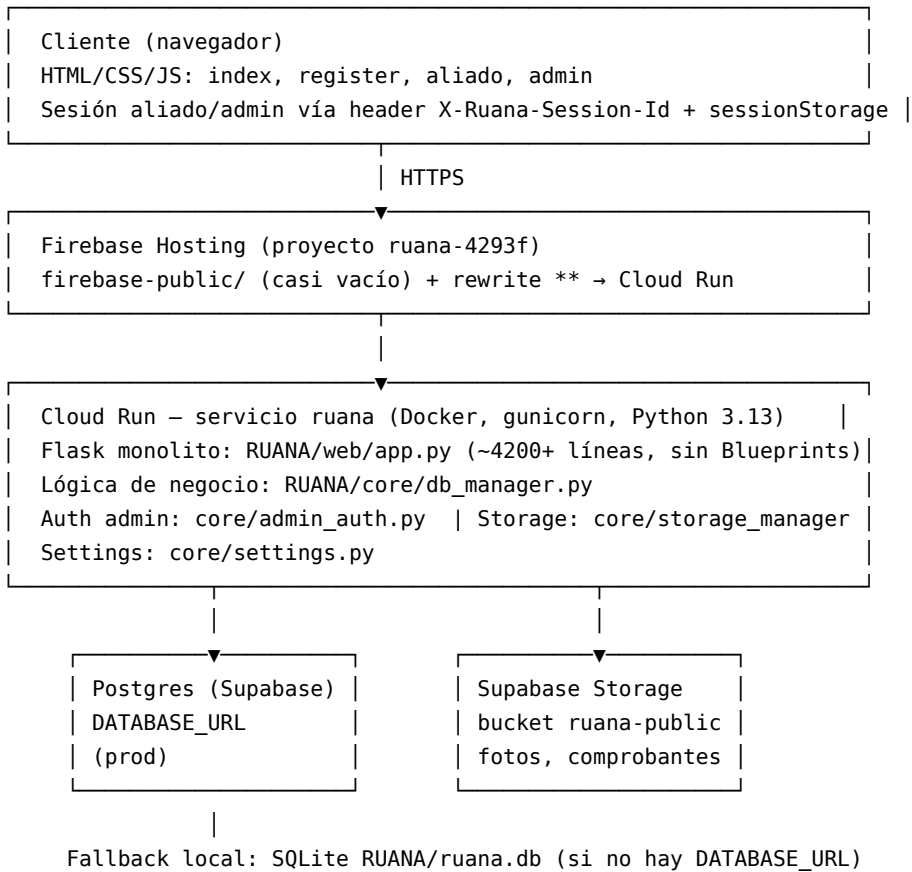
- Entrada a la red por invitación (código de aliado, invitación simple, invitación por oficio, o campaña admin).
- Un oficio principal por plaza y grupo; máximo 5 grupos activos por código postal.
- Encargos entre aliados vía contacto RUANA + chat interno + cierre por importe.
- Reputación vía score 0-100 con tope ± 10 puntos/día.
- Bajo umbral de score $\rightarrow$ competencia por la plaza.
- Calidad del pool: purga mensual (competencias vencidas + suspensión temporal por inactividad/score bajo).

2.3 Objetivos del producto

1. Que contratar dentro de la red sea más seguro que fuera.
2. Que el comportamiento (respuesta, cierre, pago, presencia) tenga efecto medible.
3. Que la administración pueda auditar chats, pagos, conflictos, linaje y plazas.
4. Escalar infra (Cloud Run + Supabase) sin abandonar el modelo de reglas.

3. Arquitectura

3.1 Vista completa



Motor de evaluación (pipeline paralelo)

core/orquestador.py $\rightarrow$ metrics/collector.py $\rightarrow$ engines/motor_evaluacion.py
 $\rightarrow$ events/event_bus.py $\rightarrow$ logs/eventos_ruana.jsonl

Este pipeline no define el semáforo visible del aliado. El estado de panel es solo el derivado de aliados.score. El motor verde/amarillo/rojo es métrica interna/histórica (evaluaciones).

3.2 Componentes

Componente	Ubicación	Responsabilidad
------------	-----------	-----------------

Frontend HTML	RUANA/web/*.html	UI aliado, admin, registro, acceso
Estáticos	RUANA/web/static/	CSS premium, ruana-ui.js, admin-shell.js
API Flask	RUANA/web/app.py	Rutas HTTP, validación, auth decorators
DBManager	RUANA/core/db_manager.py	Schema, migraciones, reglas de score/grupos/chat/pagos
Admin auth	RUANA/core/admin_auth.py	
Settings	RUANA/core/settings.py	Credenciales hasheadas, permisos
Storage	RUANA/core/storage_manager.py	Env vars tipadas
Supabase client	RUANA/core/supabase_client.py	Subidas a Supabase Storage (+ fallback local)
Postgres compat	RUANA/core/postgres_compat.py	Cliente service role
Reglas config	RUANA/config/ruana_reglas_v1.json	Adaptador SQLite→Postgres
Catálogo oficios	RUANA/config/oficios_ruana.json	Umbral, Apoyo %, métodos de pago
E2E	e2e/	Oficios + especializaciones
Migraciones SQL	supabase/migrations/	Playwright flujos críticos
CI/CD	.github/workflows/, scripts/	Schema Postgres
		QA, deploy Cloud Run/Firebase

3.3 Tecnologías utilizadas

Capa	Tecnología
Frontend	HTML5, CSS (temas premium), JavaScript vanilla, Lucide icons, Plus Jakarta Sans
Backend	Python 3.13 (Docker), Flask 2.3, Flask-Cors, PyJWT, gunicorn, Werkzeug
Datos	Postgres/Supabase (prod); SQLite (local/tests)
Ficheros	Supabase Storage (ruana-public)
Auth	Sesiones propias JWT/HS256 + header X-Ruana-Session-Id; admin con password hash
Hosting	Firebase Hosting → rewrite a Cloud Run (europe-west1)
Ops	Docker, Artifact Registry, GCP Secret Manager, GitHub Actions
QA	pytest, Playwright

3.4 Relaciones entre sistemas

Sistema	Uso real en código (julio 2026)
Firebase	Solo Hosting + FIREBASE_PROJECT_ID para URL pública. Sin Firebase Auth ni Firestore en el backend/frontend actual. Plan documentado: migrar admin a Firebase Auth.
Supabase Postgres	BD principal cuando existe DATABASE_URL.
Supabase Storage	Fotos de perfil, comprobantes, QR Revolut.
SQLite	Fallback si no hay DATABASE_URL; usado en E2E/local.
Cloud Run	Ejecuta el contenedor Flask.

3.5 Configuración operativa de reglas (ruana_reglas_v1.json)

Valores actuales leídos por el código:

Clave	Valor actual	Uso
umbral_competencia	15	Score por debajo → proceso de competencia
score_reinicio_competencia	50	Score tras 1ª derrota
duracion_competencia_dias	30	Duración del reto
purga_mensual_meses_sin_ganar	3	Pool sin victoria
purga_score_bajo_umbral	40	Score bajo en purga
apoyo_pct	12.0	% Apoyo RUANA
posponer_horas	24	«Sigue en conversación»
motor_delta_*	+5 / -3 / -8	Deltas del motor evaluación
bizum_num, iban, qr_*	(config)	Métodos de cobro del Apoyo

4. Estructura del repositorio

```
/workspace
├─ README.md                # ← ESTE Manual Maestro (fuente de verdad)
├─ README_RUANA_COMPLETO.md # Copia idéntica del Manual Maestro
├─ docs/
│   ├── README.md           # Índice de docs secundarias
│   ├── INFORME_REORGANIZACION_DOCS.md
│   ├── seguridad/          # Auth, credenciales admin
│   ├── operaciones/        # Roadmap vivo, despliegue
│   └─ flujos/              # Deep-dives (chat, registro...) alineados al código
```

```
| └─ referencia/                # API expandida si se necesita
| └─ qa/                        # Planes QA vigentes
| └─ exports/                   # PDF/DOCX auditoría
| └─ archive/                   # Históricos (NO borrados)
└─ RUANA/                       # Código de aplicación
    └─ README.md                # Puntero al Manual Maestro
    └─ core/                    # db_manager, auth, storage, settings...
    └─ web/                     # Flask + HTML/CSS/JS
    └─ config/                  # reglas, oficios, credenciales ejemplo/QA
    └─ engines/, metrics/, events/
    └─ scripts/, tests/, docs/  # docs/ = punteros
    └─ ruana.db                 # SQLite local (no es fuente prod)
└─ supabase/migrations/
└─ e2e/, scripts/, firebase-public/, .github/
└─ Dockerfile, firebase.json, package.json, playwright.config.js
└─ .env.example
```

5. Reglas del negocio

Todas las reglas siguientes están implementadas en RUANA/core/db_manager.py y/o RUANA/web/app.py, salvo donde se indique lo contrario.

5.1 Score RUANA (0-100)

- Campo: aliados.score.
- Auditoría: score_movimientos (codigo_aliado, delta, motivo, creado_en).
- Rango forzado: [0, 100].
- Límite diario: máximo ±10 puntos netos por aliado y día calendario del servidor.
- Al registrarse un aliado nuevo, el score inicial en el flujo de registro es 50 (estable).
- El estado visible se calcula con DBManager.score_a_estado(score) (no se almacena como columna de estado RUANA):

Score	Estado RUANA
85-100	DESTACADO
50-84	ESTABLE
15-49	EN RIESGO
0-14	COMPETENCIA

> El semáforo verde/amarillo/rojo del motor de evaluación no es el estado de panel.

5.2 Recompensas de score (Reglas 1-9)

#	Motivo típico	Disparador	Delta	Quién	
1	aliado_referido_registro_valido	Consumir invitación simple al registrarse	+3	Invitador	
2	encargo_completado_a_poyo_pagado	Admin marca Apoyo como pagado	+2	Solicitante y profesional	
3	`referido_encargo_completado_gen{1\`	2}`	Tras Regla 2	+1	Padre (gen1) y abuelo (gen2) de cada participante vía invitado_por_codigo
4	regla4_4_encargos_mes_limpio_YYYY-MM	4 encargos pagado en el mes sin incidencias de pago	+3	Aliado (una vez/mes)	
5	regla5_3_clientes_respuesta_1h...	Profesional responde <1 h al 1.er mensaje de 3 clientes distintos	+3	Profesional	
6	regla6_urgente_mismo_dia_{id}	Contacto es urgente y Apoyo pagado el mismo día calendario	+3	Profesional	
7	regla7_declaracion_24h_{id}	Contratante declara importe <24 h desde creado_en	+2	Solicitante	

8	regla8_racha_7dias_YY YY-MM-DD	Login (POST /api/aliado/login) 7 días consecutivos	+3	Aliado	
9	invitacion_oficio_usada	Se usa invitación por oficio RUANA-...	+5	Quien generó la invitación	

Nota: cerrar el contacto con importe genera Apoyo (pendiente_pago) pero no suma score por sí solo. Los puntos de Regla 2 llegan al confirmar el pago (pagado).

5.3 Penalizaciones de score

#	Motivo / tipo	Condición	Delta	
1	contacto_cerrado_no_concre tado	Cierre sin trabajo concretado	-1 solicitante y -1 profesional	
2	contacto_sin_cerrar_7d	Abierto ≥7 días	-2 (una vez)	
3	contacto_sin_cerrar_21d	Abierto ≥21 días	-5 (una vez; se suma al -2)	
4	`descendiente_entra_compe tencia_gen{1\`	2}_...`	Hijo/nieto entra en competencia real	-2 padre y/o abuelo
5	chat_sin_respuesta_48h_{id }	Sin respuesta ≥48 h desde último mensaje	-2 a quien no es el último emisor	
6	sin_acceso_7d_...	Sin login 7 días (bloques repetibles)	-1 por bloque	
7	chat_agotado_sin_resultado_ {id}	Agotar 30 mensajes totales sin resultado	-2 a quien agotó el cupo	
8	disputa_perdida_{id}	Admin da la razón a la otra parte	-3 al perdedor	
9	comprobante_apoyo_3d_{id }	Apoyo sin comprobante ≥3 días	-3 profesional	

Hooks:

- No concretado → al marcar cierre.
 - 7d/21d, chat 48h, sin acceso, comprobante 3d → al pedir datos del aliado / login (antes de registrar acceso del día para sin_acceso).
 - Descendiente → al iniciar competencia.
 - Chat agotado → en enviar_mensaje_chat.
 - Disputa → en resolución admin.
- > Unificación: el docstring de registrar_importe_contacto menciona «Disputa → -1»; el comportamiento vigente no resta score al declarar discrepancia: solo al perder la disputa resuelta por admin (-3). El código de cierre actual solo permite que declare el solicitante (contratante); al declarar, si no hay importe profesional o coincide, cierra el trabajo.

5.4 Grupos y plazas

- Grupos territoriales por codigo_postal.
- Nombre automático tipo RUANA-<ID>-<SUFILJO> (sufijos: PUENTE, FARO, NEXO, RAÍZ, PLAZA, RED, HOGAR, IMPULSO, ORIGEN, ENLACE).
- Estados de grupo: activo | en_competencia | disuelto.
- Máximo 5 grupos activos por CP.
- Una plaza por oficio principal por grupo (no se repite el mismo oficio entre activos del grupo).
- Las especializaciones del catálogo se guardan por compatibilidad / UI de registro, pero no ocupan plaza. Comentario en código: «Suboficios/especializaciones ignorados: plaza solo por oficio principal».
- Si el CP está lleno (5 grupos) y el oficio está ocupado en todos → el nuevo aliado queda en estado en_espera (lista de Suplentes), no se rechaza el registro.
- Oficio fuera de catálogo → pendiente_validacion (activación/rechazo admin).
- Viabilidad: grupo viable = mínimo 2 aliados activos. Si baja a 1: intento de fusión con otro grupo del CP (<3 aliados, sin oficios repetidos; el más antiguo absorbe) o reasignación / nuevo grupo; el grupo residual se marca disuelto. El nombre disuelto no se reutiliza.

5.5 Suplentes (en_espera)

- Lista de aliados registrados sin plaza libre en su zona/oficio.
- No pueden hacer login al panel (403 con mensaje de suplentes).

- Admin puede listar e incorporar (POST /api/admin/suplentes-espera/<codigo>/incorporar).
- En competencia, el retador preferente puede ser un suplente del mismo CP/oficio.

5.6 Competencia

1. Cuando el score cae por debajo de umbral_competencia (15), se inicia proceso de competencia (puede quedar competencia_pendiente hasta haber retador).
 2. Selección de retador: prioridad suplente en_espera mismo oficio/CP; si no, activo compatible.
 3. Duración: duracion_competencia_dias (30).
 4. Al finalizar: mayor score permanece en la plaza.
 5. Primera derrota: el perdedor no se expulsa; score se reinicia a score_reinicio_competencia (50 en config actual); se reasigna/crea grupo.
- > Docs antiguos decían reinicio a 75; código = 50.
1. Segunda derrota (derrotas_competencia >= 2): estado expulsado; el código deja de dar acceso; para volver hace falta nueva invitación/registro.
 2. Si el score se recupera antes de iniciarse la competencia pendiente, se puede cancelar la pendiente.
 3. Admin puede forzar competencia/suplencia. Cron/API: POST /api/competencia/finalizar-vencidas.

5.7 Invitaciones y referidos

Tipos:

1. Código de aliado existente (5 dígitos o legacy): el invitado se registra indicando ese código; debe estar activo o pendiente_completar (no expulsado).
2. Invitación simple (invitaciones): generada por aliado; un uso; al consumirse → Regla 1 (+3) + fila en referidos + invitado_por_codigo.
3. Invitación por oficio (invitaciones_oficio): formato RUANA-{grupo_id}-{OFICIO_NORM}-{4chars}; un uso; Regla 9 (+5).
4. Campañas admin (invitacion_campanas + invitacion_campana_usos): códigos reutilizables con tope/usos; creadas desde panel admin.

Validación pública: GET /api/validar-invitacion, /api/invitaciones/validar, /api/invitaciones/validar/<codigo>.

Linaje: árbol de referidos consultable por aliado (su rama) y por admin (global).

5.8 Solicitudes de grupo

- Tabla unificada solicitudes: grupo_id, solicitante_codigo, solicitante_nombre, oficio, descripcion, estado (pendiente|atendida), atendido_por_*, timestamps.
- Un aliado del grupo crea una solicitud de oficio/servicio; otro la atiende (o admin).
- API aliado: GET/POST /api/solicitudes, POST /api/solicitudes/<id>/atender.
- API admin: listar y atender.

5.9 Encargos (contactos RUANA)

Un contacto une solicitante y profesional.

Ciclo de estados (contacto)

iniciado → aceptado (profesional) → trabajo_en_progreso → cierre:

- trabajo_cerrado (importe confirmado por el contratante según regla vigente)
- no_concretado / cerrado_no_concretado
- importe_en_disputa (conflicto; resolución admin)
- También aparecen en flujo de UI: en_conversacion, chat_agotado

Estados finales (_ESTADOS_FINALES_CONTACTO): trabajo_cerrado, no_concretado, cerrado_no_concretado, importe_en_disputa.

Cierre económico vigente (código)

- Solo el solicitante (contratante) puede confirmar el importe.
- Al confirmar: se genera importe_final, apoyo_ruana, estado_pago=pendiente_pago, fila en ingresos_ruana,

evento/notificación.

- Si hubiera importe profesional previo distinto → disputa (rama legacy de doble declaración; la validación actual bloquea declaración del profesional).

Pagos / Apoyo

estado_pago: no_generado → pendiente_pago → en_revision (comprobante) → pagado | rechazo vuelve a pendiente_pago.

Admin: GET /api/admin/pagos-apoyo, pagos-en-revision, POST .../estado-pago.

Impugnación: POST /api/contactos/<id>/impugnar-apoyo. Conflictos: tablas/API payment_conflicts.

Métodos de cobro RUANA (no del aliado individual para Apoyo plataforma): Bizum, IBAN, QR Revolut en ruana_reglas_v1.json / endpoints admin.

5.10 Chat

Constantes en código:

- CHAT_MAX_MENSAJES_TOTAL = 30 (límite total del contacto, no 5 por usuario).

- CHAT_HORAS_VIGENCIA = 48 (desde última actividad: último mensaje o aceptación).

Al agotar el cupo → estado chat_agotado + posible penalización a quien envió el mensaje límite.

APIs: /api/chat/mensajes, /api/chat/enviar, /api/contactos/<id>/mensajes, aliases legacy chat_mensajes / chat_enviar.

> Unificación: docs antiguos (LOGICA_CHAT..., partes del README previo) decían 5 mensajes por usuario. El código vigente usa 30 mensajes totales.

5.11 Alertas de contactos abiertos

- GET /api/contactos/abiertos/<codigo>: contactos que deben mostrarse como aviso.

- «Sigue en conversación»: POST .../en-conversacion → posponer_recordatorio + fecha_pospuesto_hasta (posponer_horas, default 24).

- «Finalizar chat»: POST .../finalizar-chat → fila en contacto_panel_oculto (oculta del panel de ese aliado; no borra el contacto).

5.12 Notificaciones y centro de comunicación

- notificaciones_aliado: inbox (Apoyo RUANA, competencia, score, etc.).

- Centro de soporte: ruana_soporte_conversaciones / ruana_soporte_mensajes — aliado escribe, admin responde/cierra.

5.13 Reputación, rankings y recomendaciones

- Reputación operativa = score + estado RUANA + historial de contactos/pagos/competencias.

- Rankings implícitos: directorio de grupo, métricas admin (top invitadores, movimiento 24h), árbol de referidos.

- Recomendaciones entre aliados: el directorio y el flujo de contacto son el canal; no hay un motor ML de recomendaciones separado en el código actual.

- Motor evaluación (verde/amarillo/rojo) aporta métricas internas (tasa_respuesta, tasa_confirmacion, meses_sin_trabajo) pero no sustituye el estado de panel.

5.14 Purga mensual

- Script: RUANA/scripts/purga_mensual.py (+ API POST /api/purga/mensual).

- Finaliza competencias vencidas.

- Pool: aliados sin victoria en N meses o score < purga_score_bajo_umbral → suspendido_temporal (según implementación de purga).

- Plantilla cron: RUANA/scripts/cron_purga_mensual.txt.

5.15 Administración (reglas de gobernanza)

El admin puede: validar oficios fuera de catálogo, rechazar, eliminar, pausar, forzar competencia, incorporar suplentes, cerrar/abrir plazas, cambiar reglas, resolver conflictos de pago, confirmar Apoyo, auditar chats, gestionar campañas de invitación, métodos de pago y centro de comunicación. Detalle en §13.

6. Flujos completos

6.1 Acceso e ingreso

1. Usuario abre / (o /invite).
2. Introduce un código (ingreso o invitación).
3. Frontend llama GET /api/validar-invitation?codigo=....
4. Si es invitación válida → register.html (guarda datos en sessionStorage).
5. Si es aliado existente → POST /api/aliado/login → recibe session_id → guarda en sessionStorage → redirige a /aliado.

6.2 Registro de aliado

1. Formulario: nombre, email, teléfono, CP, oficio principal (+ especializaciones UI), condiciones, codigo_invitacion opcional.
 2. Validaciones F07: nombre ≥ 3 , email con @ y ., teléfono ≥ 7 dígitos, unicidad email/teléfono.
 3. POST /api/aliados/registrar.
 4. Backend genera código de 5 dígitos; score inicial 50.
 5. Asignación de grupo / en_espera / pendiente_validacion según catálogo y plazas.
 6. Si hay invitación consumible → recompensa Regla 1 o 9 + linaje.
 7. UI muestra el código → login → panel.
- Detalle ampliado (alineado a código): docs/flujos/registro-aliados.md.

6.3 Encargo completo (contacto → pago)

Directorio → crear contacto (iniciado)

- profesional acepta (aceptado)
- chat (máx 30 msgs / 48 h)
- trabajo_en_progreso (opcional en flujo)
- contratante declara importe
 - └ cierre trabajo_cerrado + Apoyo pendiente_pago
 - └ (rama disputa si aplica)
- profesional sube comprobante (en_revision)
- admin marca pagado → Reglas score 2/3/4/6

Alternativas: no_concretado (-1/-1); posponer alerta; finalizar chat (ocultar panel).

6.4 Chat y alerta

1. Tras aceptación, ambas partes usan el chat del contacto.
 2. Cada mensaje cuenta para el tope total 30.
 3. Si no hay respuesta 48 h → penalización al silencioso (al refrescar datos).
 4. Alerta de abiertos en panel; puede posponerse 24 h o ocultarse por aliado.
- Detalle: docs/flujos/chat-y-alerta.md.

6.5 Competencia

1. Score ≤ 15 → pendiente/inicio.
2. Retador entra; aviso al grupo.
3. 30 días de convivencia de scores.
4. Finalización automática o finalizar-vencidas.
5. Ganador permanece; perdedor: reinicio 50 o expulsión a la 2ª.

6.6 Solicitud de oficio en el grupo

1. Aliado crea solicitud (POST /api/solicitudes).
2. Aparece a compañeros del grupo.
3. Otro aliado (o admin) atiende.
4. Puede derivar en contacto/invitación de oficio según contexto de UI.

6.7 Invitación por oficio faltante

- 1. Aliado ve oficios faltantes en su grupo.
- 2. Genera código RUANA-... (POST /api/generar-invitacion).
- 3. Nuevo profesional se registra con ese código → plaza de ese oficio → +5 al generador.

6.8 Campaña de invitación admin

- 1. Admin crea campaña (código reutilizable, límites).
- 2. Varios registros consumen usos.
- 3. Admin puede desactivar la campaña.

6.9 Login diario y racha

- 1. Cada POST /api/aliado/login registra día en aliado_accesos_dia (tras aplicar penalización por ausencia).
- 2. 7 días seguidos → +3 (Regla 8).
- 3. Cada bloque de 7 días sin acceso → -1.

6.10 Purga mensual

- 1. Cron día 1 o llamada admin.
- 2. Finaliza competencias vencidas.
- 3. Aplica reglas de pool (suspensión temporal).

7. Roles

Rol	Autenticación	Capacidad principal
Público	Ninguna	Validar invitación, registrar, ver catálogo/health, login endpoints
Aliado	POST /api/aliado/login → X-Ruana-Session-Id	Panel, contactos, chat, solicitudes, invitaciones, perfil, pagos pendientes, linaje propio
Administrador	POST /api/admin/validar (id + password) → sesión/JWT	Dashboard, gobernanza, pagos, conflictos, chats, reglas, campañas
Admin lectura	Mismos endpoints; permisos sin escribir/configurar	Ver paneles; mutaciones → 403
Admin escritura	Permiso escribir o configurar	Mutaciones (@require_admin_escritura)
Sistema	N/A (actor en audit/eventos)	Nodo de referidos admin/sistema; no es usuario de panel

El aliado solo puede editar un subconjunto de campos propios (_ALIADO_SELF_EDITABLE_FIELDS en app.py). Datos sensibles de otro código: admin o self.

8. Estados

8.1 Estados de aliado (aliados.estado)

Estado	Significado
activo	Operativo en panel (si no hay otras restricciones)
pendiente_validacion	Oficio fuera de catálogo; espera admin
pendiente_completar	Registro incompleto (compat)
en_espera	Suplente sin plaza; sin acceso al panel
rechazado	Rechazado por admin; sin acceso
expulsado	2ª derrota u expulsión; código inválido para acceso
suspendido_temporal	Purga / sanción temporal; login denegado
sistema	Nodo especial de linaje/admin

Login deniega entre otros: expulsado, pendiente_validacion, rechazado, suspendido_temporal, en_espera.

8.2 Estado RUANA derivado (score)

Ver §5.1: DESTACADO / ESTABLE / EN RIESGO / COMPETENCIA.

8.3 Estados de grupo

activo | en_competencia | disuelto

8.4 Estados de solicitud

pendiente | atendida

8.5 Estados de contacto

Operativos: iniciado, aceptado, trabajo_en_progreso, en_conversacion, chat_agotado. Finales: trabajo_cerrado, no_concretado, cerrado_no_concretado, importe_en_disputa.

8.6 Estados de pago

no_generado | pendiente_pago | en_revision | pagado | (rechazo operativo vuelve a pendiente_pago; admin acepta rechazado en endpoint de estado-pago)

8.7 Competencia

- competencia.estado: activa | finalizada
- competencia_pendiente.estado: pendiente | cancelada | iniciada

8.8 Conflictos de pago

Estados usados en flujo: PENDIENTE_PRUEBA, EN_REVISION, más resolución admin.

9. Base de datos

9.1 Motor

- Producción: Postgres vía DATABASE_URL (pooler Supabase recomendado).
- Local/tests: SQLite (RUANA_DB_PATH o RUANA/ruana.db).
- Schema creado/migrado por DBManager (SQLite) y por supabase/migrations/*.sql (Postgres).
- No hay colecciones Firestore. El dominio de negocio vive en tablas SQL.

9.2 Tablas principales y campos

aliados

id, codigo (único), nombre, marca, oficio, codigo_postal, email, telefono, estado, score, grupo_id, derrotas_competencia, especializaciones / especializacion (legacy/compat), descripcion_servicio, foto_perfil_url, qr_paypal_path, bizum_num, invitado_por_codigo, creado_en, actualizado_en.

grupos

id, nombre (único), codigo_postal, ciudad, provincia, estado ∈ {activo, en_competencia, disuelto}, fecha_creacion.

solicitudes (schema unificado)

id, grupo_id, solicitante_codigo, solicitante_nombre, oficio, descripcion, estado, atendido_por_codigo, atendido_por_nombre, created_at, atendido_at.

contactos_ruana

Identidad del encargo: solicitante_codigo, profesional_codigo, servicio, motivo_contacto, es_urgente, estado. Importes: importe_solicitante*, importe_profesional*, importe_final, apoyo_ruana, comision*. Pago: estado_pago, pendiente_pago, comprobante_ruta, flags antifraude. Fechas de ciclo + posponer_recordatorio, fecha_pospuesto_hasta, metadata.

chat_mensajes

id, contacto_id, emisor_codigo, receptor_codigo, texto, creado_en.

contacto_panel_oculto

PK (contacto_id, codigo_aliado) — «Finalizar chat» por aliado.

confirmaciones_trabajo

Una declaración de importe por aliado y contacto.

ingresos_ruana

contacto_id, importe_final, apoyo_ruana_2pct (nombre histórico; valor = Apoyo real).

payment_conflicts

Disputas de importes / pruebas.

score_movimientos

Auditoría de deltas + base del tope diario.

contacto_penalizaciones_aplicadas

Idempotencia de penalizaciones por contacto/tipo.

Invitaciones

- invitaciones — simple, un uso.
- invitaciones_oficio — por oficio/grupo.
- invitacion_campanas + invitacion_campana_usos — campañas admin.

referidos

Linaje codigo_referido ← codigo_invitador (complementa aliados.invitado_por_codigo).

Competencia

- competencia (titular, retador, scores snapshot, fechas, ganador...).
- competencia_pendiente.

avisos_grupo, grupo_oficio_cerrado

Avisos y plazas cerradas por admin.

Evaluación

evaluaciones, evaluaciones_historico — motor interno.

Notificaciones / soporte

notificaciones_aliado, ruana_soporte_conversaciones, ruana_soporte_mensajes.

aliado_accesos_dia

Racha de login (Regla 8 / penalización ausencia).

Trazabilidad

eventos_sistema, audit_log, migraciones.

9.3 Migraciones Postgres relevantes

Ver supabase/migrations/ (init limpio, compat SQLite, foto perfil, linaje, urgente, accesos día, plazas/suplentes, purge placeholders, etc.).

10. API

Base local: http://127.0.0.1:5000. Auth: header X-Ruana-Session-Id (aliado/admin). Admin también puede usar Authorization: Bearer <JWT>.

Leyenda de auth: Público | Aliado | Admin | AdminEscritura | Self/Admin (comprobado en handler).

10.1 Páginas HTML (Público)

Métodos	Ruta	Archivo / notas
GET	/, /dashboard	index.html (acceso)
GET	/invite, /invite.html	invite.html
GET	/register, /register.html	register.html
GET	/aliado, /aliado.html, /panel	aliado.html
GET	/private-panel, /private-panel.html	sirve panel / legacy
GET	/admin	admin.html
GET	/dashboard.html	redirige / legacy
GET	/test-panel, /diagnostico-panel, /test-simple, /panel-test	diagnóstico
GET	/static/<path>	estáticos

10.2 Auth aliado

Método	Ruta	Auth	Función
POST	/api/aliado/login	Público	Login por código; Regla 8 / penalización ausencia
GET	/api/aliado/sesion	Header aliado	Comprueba sesión
POST	/api/aliado/logout	Público*	Invalida session_id
GET/POST	/api/aliado/datos	Aliado	Perfil + métricas; aplica penalizaciones

*Invalida el id enviado; no exige sesión válida previa.

10.3 Aliados / perfil / notificaciones

Método	Ruta	Auth
POST	/api/aliados/registrar	Público
GET	/api/aliados/obtener-por-codigo/<codigo>	Self/Admin (handler)
GET	/api/aliados/verificar-codigo/<codigo>	Admin
GET	/api/aliados/listar	Admin
GET	/api/aliados	Admin
GET	/api/aliados/directorio	Aliado
GET	/api/aliados/<id>	Admin
GET	/api/aliados/por-codigo/<codigo>	Aliado
PUT	/api/aliados/<codigo>	Aliado (campos self)
POST/DELETE	/api/aliados/<codigo>/foto-perfil	Aliado
GET	/api/aliados/<codigo>/notificaciones	Aliado
POST	.../notificaciones/marcar-todas-leidas	Aliado
POST	.../notificaciones/<id>/leida	Aliado
GET/POST	.../centro-comunicacion	Aliado
GET/POST	.../centro-comunicacion/<id>/mensajes	Aliado
POST	.../centro-comunicacion/<id>/marcar-leida	Aliado
POST	/api/aliado/pausar	AdminEscritura

10.4 Catálogo / health / filtros

Método	Ruta	Auth
GET	/api/catalogo/oficios, /api/catalogo/oficios-raw	Público
GET	/api/grupos/especializaciones-disponibles	Público (deprecado para plaza)
GET	/api/filtros	Público
GET	/api/health	Público
GET	/api/contactos/metricas	Público

10.5 Stats (Admin)

GET /api/stats, /api/movimiento-24h, /api/movimiento-24h-horas, /api/metricas-salud, /api/eventos-recientes.

10.6 Solicitudes

Método	Ruta	Auth
GET/POST	/api/solicitudes	Aliado
POST	/api/solicitudes/<id>/atender	Aliado
GET	/api/admin/solicitudes	Admin
POST	/api/admin/solicitudes/<id>/atender	AdminEscritura

10.7 Invitaciones

Método	Ruta	Auth
POST	/api/generar-invitation, /api/aliado/generar-invitation	Aliado
POST	/api/invitaciones/crear	Aliado
GET	/api/validar-invitation, /api/invitaciones/validar, /api/invitaciones/validar/<codigo>	Público
POST	/api/admin/invitaciones/crear	AdminEscritura
GET/POST	/api/admin/invitacion-campanas	Admin / AdminEscritura
POST	/api/admin/invitacion-campanas/<codigo>/desactivar	AdminEscritura

10.8 Contactos, chat, pagos aliado

Método	Ruta	Auth
POST	/api/contactos	Aliado
POST	.../aceptar, .../trabajo-en-progreso, .../no-concretado, .../en-conversacion	Aliado
POST	.../finalizar-chat (+ alias _)	Aliado
GET/POST	.../mensajes	Aliado
GET	/api/chat_mensajes, /api/chat/mensajes	Aliado
POST	/api/chat_enviar	Sesión comprobada en handler
POST	/api/chat/enviar	Aliado
POST	.../declarar-importe	Aliado (solo solicitante)
GET	/api/contactos/abiertos/<codigo>	Aliado
GET	/api/contactos/<id>	Aliado
GET	/api/aliado/contactos-pago-pendiente	Aliado
POST	.../comprobante-apoyo, .../impugnar-apoyo	Aliado
GET	/api/metodos-pago	Aliado
GET	/api/conflictos/por-trabajo/<id>	Aliado
POST	/api/conflictos/<id>/subir-prueba	Aliado

10.9 Referidos / linaje

Aliado: /api/aliado/referidos, .../hijos/<codigo>, .../raiz, .../cambios, /api/aliado/linaje/hijos. Admin: /api/admin/referidos/*, /api/admin/aliados/<codigo>/linaje.

10.10 Evaluaciones

Método	Ruta	Auth
GET	/api/evaluaciones/<codigo>	Aliado
GET	/api/evaluaciones/<codigo>/historico	Self/Admin en handler
GET	/api/evaluaciones, /api/evaluaciones/estadisticas	Admin
GET	/api/admin/evaluaciones/<codigo>	Admin

10.11 Competencia y purga

Método	Ruta	Auth
POST	/api/competencia/finalizar-vencidas	AdminEscritura
POST	/api/purga/mensual	AdminEscritura

10.12 Admin — auth y dashboard

Método	Ruta	Auth
POST	/api/admin/validar	Público
POST	/api/admin/logout	Público*
GET	/api/admin/me	Admin
POST	/api/admin/cambiar-contraseña	Admin
GET	/api/admin/health-metrics, /stats-24h, /invitaciones-recientes, /dashboard-summary	Admin

10.13 Admin — operaciones

Incluye (todas Admin o AdminEscritura según mutación): forzar competencia/suplencia; suplentes espera + incorporar; pending users / activar / rechazar / eliminar; cerrar oficio / abrir plaza / oficios cerrados; generar reporte; cambiar reglas; métodos de pago (+ QR Revolut); payment-conflicts CRUD resolución;

conversaciones/chats/mensajes; competencias activas/pendientes/historial; centro-comunicación; pagos-apoyo / pagos-en-revision / estado-pago.

Lista exhaustiva alineada a app.py (143 grupos de rutas). Cualquier ruta nueva debe documentarse aquí al implementarse.

11. Seguridad

11.1 Autenticación aliado

- Login con código personal (sin password).
 - Sesión firmada (JWT HS256 con FLASK_SECRET_KEY) transportada en X-Ruana-Session-Id.
 - El frontend guarda el id en sessionStorage (por pestaña) para evitar sesiones cruzadas entre pestañas del mismo origen.
 - Expiración: RUANA_ALIADO_SESSION_EXPIRES (default 3600 s).
- Detalle técnico: docs/seguridad/autenticacion-sesiones.md.

11.2 Autenticación admin

- Identificador + contraseña (hashes Werkzeug).
 - Fuentes: RUANA_ADMIN_CREDENTIALS_JSON, RUANA_ADMIN_CREDENTIALS_PATH, legacy admin_codes.json (migración), QA admin_credentials.qa.json.
 - No usar códigos en claro tipo ADMIN001/0000 en producción; el README antiguo que los citaba como vigentes está archivado.
 - Tras validar: sesión + JWT opcional (Authorization: Bearer).
 - Permisos: leer, escribir, eliminar, configurar. Escritura exige escribir o configurar.
 - Middleware: casi todo /api/admin/* exige admin excepto /validar y /logout.
 - Bypass por URL (?bypass=) no tiene efecto.
- Setup producción: docs/seguridad/credenciales-admin.md. Plan futuro Firebase Auth: docs/archive/superpowers/plans/2026-07-27-admin-firebase-auth-migration.md.

11.3 Autorización

- Decoradores: @require_aliado, @require_admin, @require_admin_escritura.
- Self-or-admin en lecturas sensibles.
- Uploads: límite ~2 MB general / foto de perfil con optimización (hasta ~15 MB de entrada según handler).
- Superficie pública restante (registro, validar invitación, health, catálogo, métricas contactos, chat_enviar con check manual) — parte del backlog de Hito 2 de seguridad.

11.4 Datos y secretos

- Secretos solo por entorno / Secret Manager; ver .env.example.
- No commitear admin_codes.json real ni claves Supabase.

12. Variables de entorno

Variable	Uso
FLASK_SECRET_KEY	Firma JWT/sesiones (cambiar en prod)
DATABASE_URL	Postgres/Supabase (si vacío → SQLite)
RUANA_DB_PATH	Ruta SQLite fallback
SUPABASE_URL	API/Storage
SUPABASE_SERVICE_ROLE_KEY	Backend (service role)
SUPABASE_ANON_KEY	Cliente público / RLS (frontend futuro)
FIREBASE_PROJECT_ID	Default ruana-4293f; URL pública
RUANA_PUBLIC_APP_URL / PUBLIC_APP_URL	URL pública explícita
GOOGLE_CLOUD_REGION	Default europe-west1
ARTIFACT_REGISTRY_REPOSITORY	Deploy (scripts npm)
RUANA_ADMIN_SESSION_EXPIRES	Segundos (default 3600)
RUANA_ALIADO_SESSION_EXPIRES	Segundos (default 3600)
RUANA_ADMIN_CREDENTIALS_PATH	JSON admins en disco

Carga local opcional: .env.local / .env en la raíz del repo (python-dotenv vía settings.py).

13. Panel de administración

UI: RUANA/web/admin.html + static/js/admin-shell.js + CSS admin-premium.

Secciones típicas del shell:

1. Resumen / Movimiento 24h / Métricas de salud
2. Pendientes de validación (activar/rechazar)
3. Conflictos de pago
4. Pagos Apoyo / en revisión
5. Solicitudes
6. Competencias (activas, pendientes, historial; forzar)
7. Suplentes en espera (incorporar)
8. Registro de chats / conversaciones
9. Centro de comunicación (soporte)
10. Control de aliados + linaje
11. Trazabilidad / eventos
12. Métodos de pago (Bizum, IBAN, QR Revolut)
13. Acciones: reglas, purga, reportes, plazas, invitaciones/campañas, cambio de contraseña

Permisos de lectura vs escritura controlan botones y APIs.

14. Frontend

Pantalla	Ruta	Archivo	Rol
Acceso	/, /invite	index.html / invite.html	Código ingreso o invitación
Registro	/register	register.html	Alta aliado
Panel aliado	/aliado	aliado.html	Operación diaria
Admin	/admin	admin.html	Gobernanza
Legacy	dashboard.html, private-panel*.html	Redirigen o residuales	No son fuente de verdad UI

Stack UI: JS vanilla, CSS premium, Lucide (aliado), ruana-ui.js (toasts/diálogos). No hay SDK Firebase en el cliente. Auth = sessionStorage + header.

Panel aliado incluye: alertas, pagos pendientes, perfil, competencia, grupo/oficios, métricas, directorio, solicitudes, chat, importe, Apoyo, linaje, notificaciones, centro de mensajes.

15. Despliegue y operaciones

- Docker: Dockerfile → gunicorn web.app:app puerto 8080.
 - Firebase Hosting: firebase.json reescribe todo a Cloud Run servicio ruana.
 - Scripts npm: deploy Firebase/Cloud Run, secrets GCP, verify Supabase/runtime.
 - QA: Playwright (e2e/), pytest (RUANA/tests/), workflow ruana-qa.yml.
 - Cron purga: RUANA/scripts/purga_mensual.py.
 - Realtime esperado (ops): tablas chat_mensajes, notificaciones_aliado, contactos_ruana (ver scripts verify).
- Operaciones vivas: docs/operaciones/roadmap.md.

16. Cómo ejecutar

Requisitos

- Python 3.11+ (Docker usa 3.13).
- pip install -r RUANA/web/requirements.txt (y requirements-dev.txt para tests).
- Opcional: Node para Playwright/npm scripts; Docker; cuentas Firebase/Supabase/GCP para prod.

Local (SQLite)

```
cd RUANA/web
pip install -r requirements.txt
python run.py
# http://127.0.0.1:5000

Semilla:

python RUANA/scripts/seed_aliados.py

Orquestador demo:

python RUANA/core/orquestador.py
```

Con Postgres

Definir DATABASE_URL, SUPABASE_* según .env.example, aplicar migraciones Supabase, y arrancar igual el Flask app.

16.1 Límites y riesgos

- Seguridad: la configuración por defecto no es endurecida para Internet abierto; usar secretos fuertes, HTTPS (Firebase/Cloud Run) y completar el cierre de endpoints públicos del Hito 2.
- Auth aliado: acceso por código sin password — el código *es* la credencial; perderlo = perder acceso.
- Escalado: SQLite solo para local/tests; producción debe usar Postgres (DATABASE_URL).
- Datos personales: historial completo y trazable; considerar GDPR/privacidad en producción.
- Motor de evaluación: pipeline paralelo con métricas que pueden ser de ejemplo en demo; no sustituye el score de panel.
- Firebase Auth admin: aún no implementado; puente por secretos hasheados.
- Docs vs código: ante duda, leer db_manager.py / app.py y actualizar este Manual.

17. Roadmap

Estado operativo resumido (julio 2026). Detalle vivo: docs/operaciones/roadmap.md. Histórico: docs/archive/ROADMAP_2026-05.md.

Hito	Estado	Notas
1 — Auditoría e infra (Supabase/Firebase/Cloud Run)	Cerrado documentalmente	Base de despliegue
2 — Seguridad y permisos	Activo / parcialmente avanzado	2A endpoints críticos; quedan endurecimientos
2B+ — Admin invitaciones, campañas	Avanzado en código	Docs de idea archivados como históricos
Storage métodos de pago	Implementado en código	Plan original en archive
Impugnación / alertas cobros	Implementado con tests	Plan en archive
Competencia automática por score	En main (jul 2026)	Umbral 15, reinicio 50
Migración admin → Firebase Auth	Preparado, no implementado	Plan en archive
Hitos posteriores (producto/escala)	Pendientes	Ver roadmap operativo

Método de trabajo: un hito activo, cambios pequeños, tests en permisos/dinero, código como verdad.

18. Historial (changelog)

Resumen de cambios importantes (no exhaustivo; basado en commits y docs):

Fecha	Cambio
2026-05	Auditoría base, migraciones Supabase iniciales, roadmap unificado, Hito 2A seguridad
2026-05-28	Admin crear código de aliado (spec/plan)
2026-06	Plan QA E2E Playwright; flujos solicitudes; storage Supabase métodos de pago; impugnación cobros
2026-06-07	Sesiones seguras por pestaña (X-Ruana-Session-Id)
2026-07	Foto perfil, linaje, contactos urgentes, accesos día, plazas/suplentes, purge placeholders
2026-07-26	Auditoría forense del repositorio
2026-07-27	Plan migración admin → Firebase Auth; puente credenciales admin
2026-07	Competencia automática / permanencia por score (umbral 15, reinicio 50)
2026-07-28	Reorganización completa de documentación; este Manual Maestro pasa a ser la única fuente de verdad

Archivo histórico completo de documentos previos: docs/archive/.

19. Índice de documentación secundaria

Documento	Rol
README_RUANA_COMPLETO.md	Copia idéntica de este Manual Maestro
docs/README.md	Índice del árbol docs/
docs/INFORME_REORGANIZACION_DOCS.md	Informe de esta reorganización
docs/seguridad/autenticacion-sesiones.md	Deep-dive sesiones
docs/seguridad/credenciales-admin.md	Setup credenciales admin prod
docs/flujos/chat-y-alerta.md	Chat/alerta alineado a código
docs/flujos/registro-aliados.md	Registro/plazas alineado a código
docs/operaciones/roadmap.md	Roadmap operativo
docs/qa/plan-testing.md	Plan QA vigente
docs/exports/	PDF/DOCX auditoría
docs/archive/	Históricos (no borrar)
RUANA/README.md	Puntero al Manual Maestro

Plantilla GitHub: `.github/ISSUE_TEMPLATE/admin-firebase-auth-migration.md`.

20. Notas de unificación documental

Durante la reorganización (2026-07-28) se detectaron y resolvieron estas inconsistencias a favor del código:

Tema	Docs antiguos	Verdad de código / config
Persistencia	«SQLite única verdad»	Postgres/Supabase en prod; SQLite fallback
apoyo_pct	15 %, 5 %, «2 %»	12.0 en ruana_reglas_v1.json
Límite chat	5 msgs/usuario	30 msgs totales
Reinicio tras 1ª derrota	75	50 (score_reinicio_competencia)
Declaración de importe	Ambas partes	Solo solicitante cierra
Auth admin	admin_codes.json / ADMIN001/0000	Credenciales hasheadas + env secrets
Plaza	Por especialización (algunos docs)	Por oficio principal
Firebase cliente	Implícito	Sin SDK; solo Hosting→Cloud Run
Campañas invitación	«Idea futura»	Implementadas en API/UI
HITOS / auditoría may-19	Citados en roadmap	Archivos ausentes → referencias archivadas
Puerto	5050 en mapa mental	5000
Fuente de verdad múltiple	README RUANA + ROADMAP + auditoría	Este README es la única fuente de verdad

Ningún documento histórico se eliminó: viven en docs/archive/ con su contenido íntegro.

Fin del Manual Maestro RUANA.